

Outline

Why and how

Filter methods

Embedded methods

Wrapper methods

Stability and pitfalls

Wrap-up

Why select features?

Five reasons to drop features:

1. **Curse of dimensionality** (math module 30). Volume of p -dim space grows exponentially. With n fixed, samples become sparse.
2. **Multicollinearity** (L01b). Correlated features make linear coefficients unstable; $\mathbf{X}^\top \mathbf{X}$ becomes ill-conditioned.
3. **Overfitting risk**. More features $\Rightarrow$ more flexibility $\Rightarrow$ higher variance. Regularization helps but doesn't eliminate.
4. **Compute and storage**. Linear-in- p at inference, polynomial during fit.
5. **Interpretability**. "We use 8 features" defends better than "we use 800". Critical for healthcare, credit, audit.

When NOT to select. With heavy L1/L2 regularization, the model effectively selects *soft*. Sometimes deleting features hurts more than it helps - the dropped features may carry small but real signal.

Predict-first: does adding features always help?

You have $n = 200$ samples. You add features one at a time, fitting a Ridge model after each addition. Each new feature is a real signal (correlated with y) plus some noise.

Predict-first: does adding features always help?

You have $n = 200$ samples. You add features one at a time, fitting a Ridge model after each addition. Each new feature is a real signal (correlated with y) plus some noise.

Predict: as you keep adding features, what happens to

- ▶ training error?
- ▶ test error?

Predict-first: does adding features always help?

You have $n = 200$ samples. You add features one at a time, fitting a Ridge model after each addition. Each new feature is a real signal (correlated with y) plus some noise.

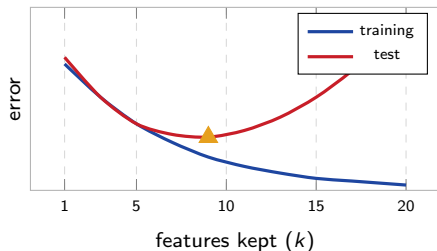
Predict: as you keep adding features, what happens to

- ▶ training error?
- ▶ test error?

Training error **always falls** - more features means more flexibility to fit the training set.

Test error follows a **U-shape**. It falls while features carry signal, then rises as added features bring more variance than bias reduction. The minimum is the sweet spot.

Feature selection is about finding that sweet spot.



Sweet spot at $k \approx 9$; more features hurt.

The three families of selection methods

Family	How it works	Strengths / weaknesses
Filter	Score each feature <i>independently</i> against the target. Pick top k .	Fast, model-agnostic. <i>Blind to interactions.</i>
Embedded	Use a model that selects <i>as it fits</i> (Lasso, tree importances).	Cheap, model-aware. Tied to the chosen model.
Wrapper	Search over feature subsets, evaluate each via CV.	Most accurate. Expensive: refits the model $\mathcal{O}(p)$ - $\mathcal{O}(p^2)$ times.

Cost goes up left-to-right. Accuracy (for the model you'll deploy) usually does too.

Typical workflow:

1. Filter from $p = 10,000$ down to $p = 500$ (drop near-constants, very low MI).
2. Embedded selection (Lasso or tree importance) on the 500 to get 50-100 candidates.
3. Wrapper (RFE-CV or Boruta) on the 50-100 to get the final 10-20.

Variance threshold and univariate filters

Variance threshold. Drop near-constant features.

Quick win - constant features carry no signal.

Univariate statistical tests. Score each feature by its individual association with y :

- ▶ Regression: `f_regression` (linear), `mutual_info_regression` (any monotonic).
- ▶ Classification: `f_classif` (ANOVA), `chi2` (categorical), `mutual_info_classif`.

Pick top k via `SelectKBest` or percentile via `SelectPercentile`.

```
from sklearn.feature_selection
import (
    VarianceThreshold,
    SelectKBest,
    f_regression,
    mutual_info_regression)

# Drop near-constants
X = VarianceThreshold(0.01).
    fit_transform(X)

# Top-20 by linear correlation
sel = SelectKBest(f_regression, k
    =20)
X_sel = sel.fit_transform(X, y)

# Top-20 by mutual information
sel_mi = SelectKBest(
    mutual_info_regression, k=20)
```

The independence trap

Filter methods score each feature *alone*. They miss **interactions**.

Consider 4 points: (x_1, x_2, y)

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

Marginal correlations: $\text{corr}(x_1, y) = 0$,
 $\text{corr}(x_2, y) = 0$. Mutual information ≈ 0 per
feature.

Jointly: $y = x_1 \oplus x_2$. Perfect predictor.

Filter score on this data: *both features look useless*. A linear model fed only x_1 or only x_2 confirms it.

Lesson: filter is a screen, not a decision. A feature that fails the univariate test may still matter in combination.

Workarounds:

- ▶ Add hand-engineered interactions before filtering (L01g).
- ▶ Use a multivariate filter (next slide).
- ▶ Use embedded methods on a tree model, which discovers interactions.

Multivariate filters

Filters that consider feature *combinations* instead of one-at-a-time.

- ▶ **mRMR** (Minimum Redundancy Maximum Relevance). At each step, pick the feature that maximizes (relevance to y) minus (average correlation with already-picked features). Catches non-redundant signal.
- ▶ **Cluster-and-represent**. Group features by mutual correlation (hierarchical clustering), pick one representative per cluster (the one with highest target correlation). Cheap, removes obvious redundancy.
- ▶ **HSIC-Lasso** (Hilbert-Schmidt Independence Criterion). Kernel-based, captures non-linear dependence.

Libraries: `pymrmr`, `mrmr-selection`, `scikit-feature`. Not as common as filter+embedded combinations, but useful when p is large and many features are correlated (genomics, sensor data).

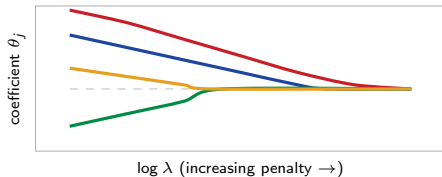
L1 / Lasso for sparsity

Recall Ridge (L01b) minimizes $\|\mathbf{y} - \mathbf{X}\boldsymbol{\theta}\|^2 + \lambda \|\boldsymbol{\theta}\|_2^2$. The L2 penalty shrinks coefficients toward zero but rarely *to* zero.

Lasso replaces L2 with L1:

$$\hat{\boldsymbol{\theta}}_{\text{Lasso}} = \arg \min_{\boldsymbol{\theta}} \|\mathbf{y} - \mathbf{X}\boldsymbol{\theta}\|^2 + \lambda \|\boldsymbol{\theta}\|_1$$

The L1 penalty has a non-smooth corner at $\theta_j = 0$, which *forces* coefficients exactly to zero. Features with zero coefficient are selected out.



Lasso path. Less-important features hit 0 first.

```
from sklearn.linear_model import
    LassoCV
from sklearn.feature_selection \
    import SelectFromModel

lasso = LassoCV(cv=5,
                 random_state=509).fit
                 (X, y)
sel = SelectFromModel(lasso,
                      prefit=True)
X_sel = sel.transform(X)
# Selected feature indices
keep = sel.get_support(indices=True)
```

Tree impurity importance

Tree models track how much each feature reduces the loss across all its splits.

- ▶ For **regression trees**: $\text{importance}_j = \text{sum of (variance reduction at each split using feature } j\text{), weighted by samples reaching that split.}$
- ▶ For **classification trees**: same idea with Gini decrease or entropy decrease.
- ▶ Random Forest / GBM average importance across all trees.

```
from sklearn.ensemble import RandomForestRegressor
rf = RandomForestRegressor(n_estimators=300,
                           random_state=509).fit(X, y)
importances = pd.Series(rf.feature_importances_, index=X.columns)
importances.sort_values(ascending=False).head(10)
```

Cheap (free with the model), intuitive, used everywhere. **Also: misleading** on correlated features and high-cardinality categoricals - the next slide is the most important warning in this deck.

Predict-first: impurity importance on correlated features

You have 5 features. Features x_1 and x_2 are nearly perfectly correlated ($\text{cor} \approx 0.99$) and both strongly predict y . Features x_3, x_4, x_5 are independent and predictive.

Predict-first: impurity importance on correlated features

You have 5 features. Features x_1 and x_2 are nearly perfectly correlated ($\text{cor} \approx 0.99$) and both strongly predict y . Features x_3, x_4, x_5 are independent and predictive.

Predict: when you fit a Random Forest and read off impurity importances, what does x_1 's importance look like?

Predict-first: impurity importance on correlated features

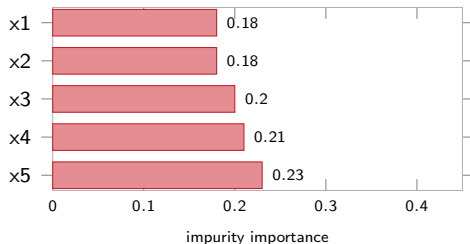
You have 5 features. Features x_1 and x_2 are nearly perfectly correlated ($\text{cor} \approx 0.99$) and both strongly predict y . Features x_3, x_4, x_5 are independent and predictive.

Predict: when you fit a Random Forest and read off impurity importances, what does x_1 's importance look like?

Each tree splits on one of $\{x_1, x_2\}$ - whichever the random feature subsample saw first. The total "true" importance is split roughly half-and-half.

A naive reader concludes: " x_1 and x_2 are about as important as x_3, x_4, x_5 . Drop x_1 to simplify."

What actually happened: x_1 alone is highly predictive; the importance is just shared with its near-duplicate. Dropping x_1 leaves x_2 to do the same work; dropping both is a disaster.



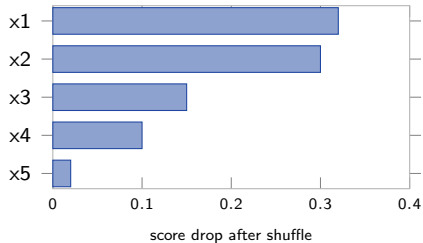
x_1 and x_2 split the importance they jointly carry.

Permutation importance: the more reliable alternative

Idea. Shuffle one column at a time; measure how much the model's score drops on a held-out set. Big drop $\Rightarrow$ feature mattered.

```
from sklearn.inspection import (
    permutation_importance)

model = RandomForestRegressor().fit(X_tr,
    y_tr)
res = permutation_importance(
    model, X_val, y_val,
    n_repeats=30,
    random_state=509)
# res.importances_mean, res.
# importances_std
```



Correlated x_1, x_2 are *both* marked important.

Advantages:

- ▶ Model-agnostic - works for any score method.
- ▶ Reflects *validation* performance, not training.
- ▶ Multiple shuffles give error bars.

Caveat: permutation breaks correlations the model relied on. If features are highly correlated, permutation importance over-credits them. The cure is to use *conditional* permutation or pair with

SHAP: model-agnostic, theoretically grounded

SHAP (SHapley Additive exPlanations) gives each feature its *Shapley value* from cooperative game theory: the average marginal contribution to the prediction across all possible feature subsets.

- ▶ Works on any model. Fast for trees via TreeExplainer (exact, polynomial-time).
- ▶ Local (per-prediction) and global (mean absolute SHAP across samples) importance.
- ▶ Modern industry default for interpretability.
- ▶ Handles correlated features more honestly than impurity importance.

```
import shap
explainer = shap.TreeExplainer(model)
shap_values = explainer.shap_values(X_val)           # one per (sample,
                                                    feature)
mean_abs_shap = np.abs(shap_values).mean(axis=0)    # global importance
shap.summary_plot(shap_values, X_val)               # built-in plot
```

Caveat: “importance” here means contribution to *the model’s prediction*, not to the true causal effect. A correlated nuisance feature can have high SHAP value if the model happens to use it.

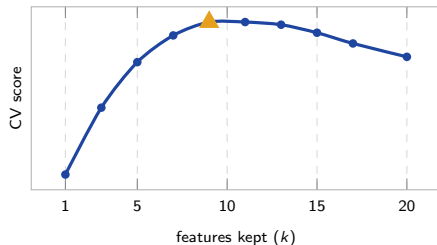
RFE and RFE-CV

Recursive Feature Elimination. Fit the model, drop the least-important feature, refit, repeat until k features remain.

```
from sklearn.feature_selection \
    import RFE, RFECV
from sklearn.linear_model import
    Ridge

# Fixed k
rfe = RFE(Ridge(),
          n_features_to_select=10).fit(X, y)

# Auto k via CV (preferred)
rfecv = RFECV(
    Ridge(), cv=5,
    scoring="neg_mean_squared_error",
    min_features_to_select=2).fit(X,
    y)
print(rfecv.n_features_)
```



Elbow at $k = 9$. RFECV picks this.

Cost: $\mathcal{O}(p)$ model fits per fold. Use a fast model (Ridge / LogReg) or pre-filter to $p < 100$.

Sequential forward / backward selection

Greedy alternatives to RFE. Slower but flexible across model classes.

```
from sklearn.feature_selection import SequentialFeatureSelector

# Forward: start empty, add the feature that most improves CV
sfs_fwd = SequentialFeatureSelector(
    Ridge(), n_features_to_select=10,
    direction="forward", cv=5).fit(X, y)

# Backward: start full, drop the feature whose removal hurts CV least
sfs_bwd = SequentialFeatureSelector(
    Ridge(), n_features_to_select=10,
    direction="backward", cv=5).fit(X, y)
```

- ▶ **Forward** - fast when target $k \ll p$ (few features kept). Each step refits with one more feature.
- ▶ **Backward** - fast when target $k \approx p$ (few features dropped). Each step refits with one fewer.
- ▶ Both miss good *combinations* that don't show in single-step additions. RFE often does better.

Boruta: the “all-relevant” approach

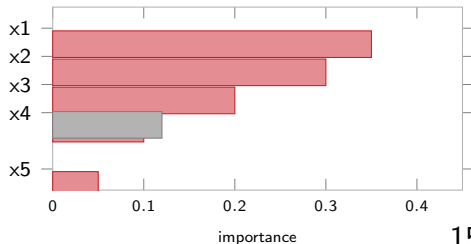
Most methods find a *minimal* relevant set. Boruta finds *all* features that beat random chance.

Algorithm.

1. For each real feature, create a *shadow*: a shuffled copy. Shadow features carry no real signal.
2. Fit a Random Forest on real + shadows.
3. Compare each real feature's importance to the *maximum* shadow importance.
4. Repeat for many iterations. A feature that consistently beats the max shadow is “confirmed”, one that consistently doesn't is “rejected”.

```
from boruta import BorutaPy
from sklearn.ensemble import \
    RandomForestRegressor

rf = RandomForestRegressor(
    n_estimators=200, random_state
    =509)
br = BorutaPy(rf, max_iter=100,
               random_state=509).fit(
    X.values, y.values)
print(X.columns[br.support_])
```



Stability selection + how many features?

“Run Lasso once” gives one answer. “Run Lasso many times on bootstrap samples” tells you which features show up *robustly*.

Stability selection (Meinshausen & Bühlmann, 2010).

1. Draw $B = 100$ bootstrap subsamples.
2. Fit Lasso (with a range of λ s) on each.
3. For each feature, record fraction of bootstraps where it was selected at all.
4. Keep features with selection frequency > 0.5 (or any chosen threshold).

```
from sklearn.utils import resample
from sklearn.linear_model import LassoCV
import numpy as np

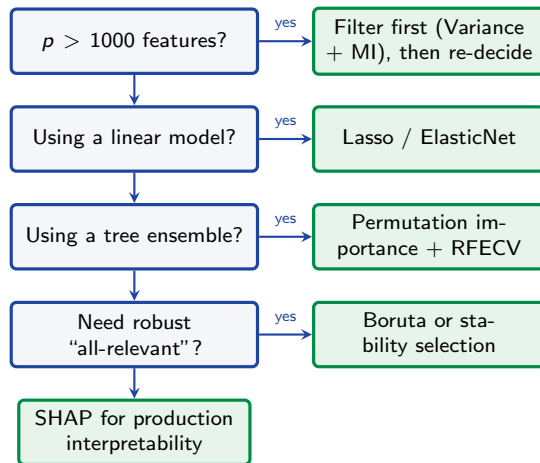
selected_counts = np.zeros(X.shape[1])
for _ in range(100):
    X_bs, y_bs = resample(X, y, random_state=None)
    sel = LassoCV(cv=5).fit(X_bs, y_bs)
    selected_counts += (sel.coef_ != 0)
stability = selected_counts / 100
keep = stability > 0.5
```

How many features? Stability selection answers it automatically - k = count of features above the threshold. Alternative: pick k at the RFE-CV elbow.

Pitfalls

1. **Selection bias (the biggest one).** Doing feature selection on the *full* dataset before splitting inflates CV scores. The chosen features were peeked at including test rows. Fix: nested CV with selection inside the inner loop, or do selection on train fold only.
2. **Leakage via target encoding.** If features were target-encoded (L01c) without CV, selection just propagates the leakage. Fix the encoder first.
3. **Model-dependence.** Features selected by Lasso may differ from those selected by tree importance. Selection results don't transfer across model classes; re-select if you switch models.
4. **Correlated features.** Impurity importance splits credit; Lasso picks arbitrarily; RFE drops one randomly. None handles correlation cleanly. SHAP and permutation help.
5. **Over-aggressive thresholds.** "Keep features with importance > 0.01 " is arbitrary. Use CV elbow, stability frequency, or Boruta's shadow comparison instead.
6. **Forgetting the baseline.** Always compare to "keep all features." Selection that doesn't improve CV *wasn't worth doing*.

Decision flowchart



Whatever method you use: do selection inside a CV loop, not before.

Recap

- ▶ **Why select.** Curse of dimensionality, multicollinearity, overfitting, compute, interpretability. Not always worth it - check vs “keep all + regularize”.
- ▶ **Filter** - cheap, model-agnostic, blind to interactions. `SelectKBest` with `f_regression` or `mutual_info_regression`.
- ▶ **Embedded** - selection as part of fitting. Lasso for linear, tree importances for trees. Watch for the correlated-features trap.
- ▶ **Permutation importance** - the reliable replacement for impurity importance. SHAP - the modern industry default.
- ▶ **Wrapper** - RFE-CV picks k automatically. Boruta finds *all relevant* features via shadow comparisons.
- ▶ **Stability selection** - bootstrap + Lasso, keep features that show up often.
- ▶ **Pitfall #1:** feature selection on full data before splitting inflates CV scores. Always select inside the CV loop.

HW01h - practice

Estimated time: 90-120 minutes. Tasks 1-4 graded; bonuses optional.

1. On the Yerevan rent dataset (with engineered features from L01g): compute `f_regression` scores. Plot scores and pick top 10 by univariate filter. Report Ridge CV MAE on the filtered set.
2. Fit LassoCV. Plot the coefficient path (`lasso_path`). Report which features survive at the CV-optimal λ and compare to step 1.
3. Fit a Random Forest. Compute *both* `feature_importances_` (impurity) and `permutation_importance` on a held-out set. Identify any feature where the two disagree by more than $2\times$. Explain why.
4. Run RFECV with Ridge. How does its chosen k compare to the filter and Lasso choices?
5. *Bonus 1*: run stability selection (bootstrap + Lasso, 100 iterations). Plot selection frequency per feature. How does it compare to a single Lasso fit?
6. *Bonus 2*: install `boruta` (`pip install Boruta`) and run `BorutaPy`. Compare confirmed features to RFE-CV's selection.
7. *Bonus 3*: simulate selection bias: do feature selection on the *full* dataset, then CV. Compare to doing selection inside the CV loop. Report the gap. **Most instructive question.**